

Robotics and Embodied AI

1 Markov Decision Processes

Dr. Armin Lederer

Department of Electrical and Computer Engineering

National University of Singapore

11th August 2026

Administrative Information

- Married course: CEG5306 and EE6108
- Lecturers: Dr Fan Shi and Dr Armin Lederer
- Grading:
 - 5 Assignments: 50% of overall grade
 - Final exam: 50% of overall grade
- Annotated slides will be uploaded to canvas



Assignment Rules

- Deadlines are strict:
 - 5% mark reduction per hour of delayed submission
→ 10h delay = -50% of achieved marks
- Post-hoc deadline extensions only in exceptional circumstances
- But: lenient when contacted before the deadline

Tentative Schedule

date	topic	lecturer	notes
11/08	Markov decision processes	Lederer	
18/08	Multi-armed bandits and model-based RL	Lederer	
25/08	Monte-Carlo and temporal difference methods	Lederer	Assign. 1
01/09	RL formulation in robotics 1	Shi	
08/09	RL formulation in robotics 2	Shi	Assign. 2
15/09	Application examples	Shi	
22/09	Reading week		

Tentative Schedule Continued

date	topic	lecturer	notes
29/09	Safety constraints in RL	Lederer	Assign. 3
06/10	Partial observability in RL	Lederer	
13/10	Learning from demonstrations	Lederer	
20/10	Deep RL 1	Shi	Assign. 4
27/10	Deep RL 2	Shi	
03/11	Current challenges in robotics	Shi	Assign. 5
27/11	Final exam (9:00 – 11:00)		

Further Reading

- R. Sutton, A. Barto: Reinforcement Learning: An Introduction (<https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>)
- A. Krause, J. Hübner: Probabilistic Artificial Intelligence (<https://arxiv.org/pdf/2502.05244>)
- Articles suggested in lectures

Word Cloud Participation

What comes to your mind when you hear Embodied Artificial Intelligence?

~~Embodied Artificial Intelligence~~



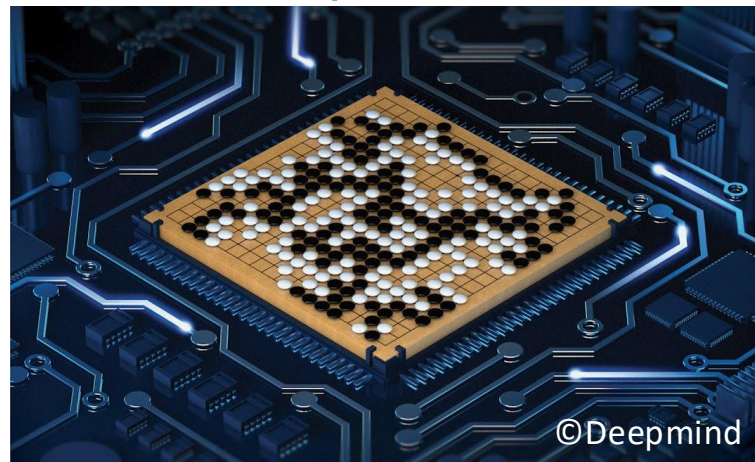
Embodied_Artificial_Intelligence

Motivation: Success in Board Games

Poker using DeepStack



AlphaGo



Human expert-like performance in board games!

Motivation: Success in Computer Games

OpenAI Five Bot for DOTA 2



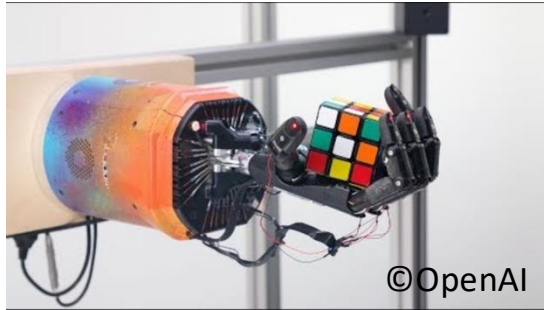
AlphaStar for StarCraft II



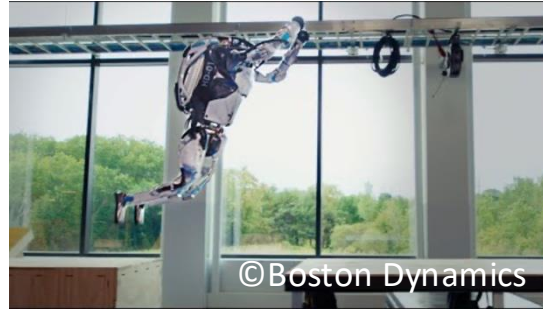
Super-human performance in complex computer games!

Motivation: Success in Robotics?

Dexterous manipul.



Humanoids



Quadrupeds



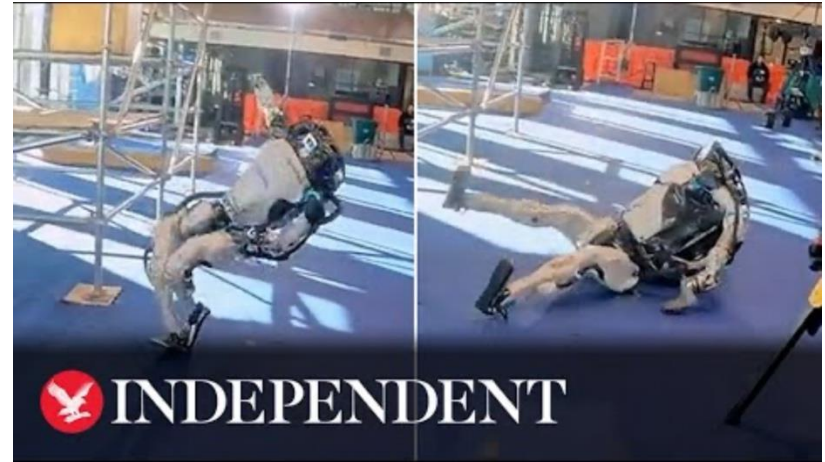
Everything solved?

Motivation: Failures of Embodied AI

Crash of autonomous vehicle



Falling humanoid



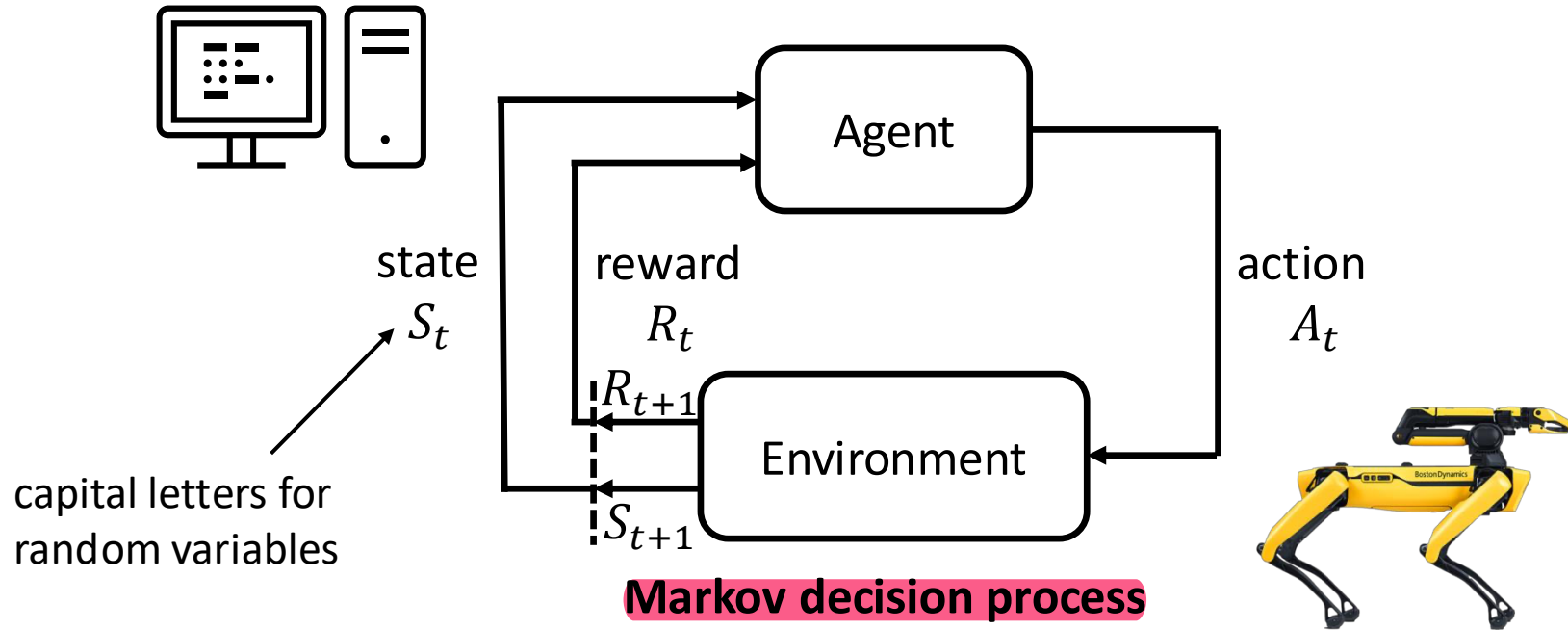
Robotics and Embodied AI not yet ready for the wild!

Learning outcomes of this course

In this course, you will learn:

- Principles of modeling robotics for AI
- Theoretical foundations of learning problems in robotics
- Practical learning algorithms
- Limitations and weaknesses of current embodied AI

The Agent-Environment Interface



Markov Decision Processes

A Markov decision process (MDP) is specified by

- A set of **states** $\mathcal{S}$
- A set of **actions** $\mathcal{A}$
- **Initial state distribution** $p(s_0)$
- **Markov kernel** $p(s'|s, a) \rightarrow$ transition model
- A **reward function** $r(s, a)$

lower case for
deterministic values

What is 'Markov' about this modelling approach?

The answer is the Markov property (马尔可夫性质):

Given the current state (S_t) and action (A_t) , the next state (S_{t+1}) is independent of the earlier history.

Example: Gymnasium Inverted Pendulum

 Gymnasium Documentation

- Make and register
- Spaces
- Wrappers
- Vectorize
- Utility functions
- Functional Env

ENVIRONMENTS

- Classic Control
 - Acrobot
 - Cart Pole
 - Mountain Car Continuous
 - Mountain Car
 - Pendulum**
- Box2D
- Toy Text
- MuJoCo
- Atari
- External Environments

TUTORIALS

- Gymnasium Basics

Pendulum

This environment is part of the Classic Control environments which contains general information about the environment.

Action Space	<code>Box(-2.0, 2.0, (1,), float32)</code>
Observation Space	<code>Box([-1. -1. -8.], [1. 1. 8.], (3,), float32)</code>
import	<code>gymnasium.make("Pendulum-v1")</code>

Description

The inverted pendulum swingup problem is based on the classic problem in control theory. The system consists of a pendulum attached at one end to a fixed point, and the other end being free. The pendulum starts in a random position and the goal is to apply torque on the free end to swing it into an upright position, with its center of gravity right above the fixed point.

The diagram below specifies the coordinate system used for the implementation of the pendulum's dynamic equations.



ON THIS PAGE

- Description
- Action Space
- Observation Space
- Rewards

Product Solutions Resources Open Source Enterprise Pricing

openai / gym Public

Code Issues Pull requests Actions Projects Wiki Security Insights

Files

- master
- github
- bin
- gym
 - envs
 - box2d
 - classic_control
 - assets
 - __init__.py
 - acrobot.py
 - cartpole.py
 - continuous_mountain_car.py
 - mountain_car.py
 - pendulum.py**
 - utils.py
 - mujooco
 - toy_text
 - __init__.py
 - registration.py
 - spaces
 - utils

gym / gym / envs / classic_control / pendulum.py

younik ENR: add render warn for None (#3112)

Code Blame 271 lines (210 loc) · 9.3 KB

```

1  __credits__ = ["Carlos Luis"]
2
3  from os import path
4  from typing import Optional
5
6  import numpy as np
7
8  import gym
9  from gym import spaces
10 from gym.envs.classic_control import utils
11 from gym.error import DependencyNotInstalled
12
13 DEFAULT_X = np.pi
14 DEFAULT_Y = 1.0
15
16
17 class PendulumEnv(gym.Env):
18     """
19     ## Description
20
21     The inverted pendulum swingup problem is based on the classic problem in control theory.
22     The system consists of a pendulum attached at one end to a fixed point, and the other end being free.
23     The pendulum starts in a random position and the goal is to apply torque on the free end to swing it
24     into an upright position, with its center of gravity right above the fixed point.
25
26     The diagram below specifies the coordinate system used for the implementation of the pendulum's
27     dynamic equations.
28
29     (./diagrams/pendulum.png)
30
31     - "tau" is the action (torque) of the pendulum and is in meter
    
```

Example: Gymnasium Inverted Pendulum

- A set of **states** $\mathbb{S} = [-1, 1] \times [-1, 1] \times [-8, 8]$

- A set of **actions** $\mathbb{A} = [-2, 2]$

- **Initial state distribution**

$\mathcal{U}(a, b)$ means a uniform distribution (均匀分布) between a and b .

$$P(s_0) = [\cos(\mathcal{U}(-\pi, \pi)), \sin(\mathcal{U}(-\pi, \pi)), \mathcal{U}(-1, 1)]$$

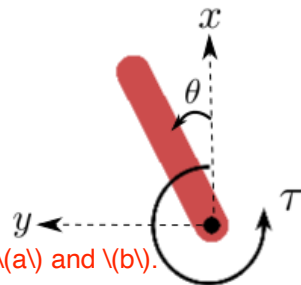
- **Transition kernel** $p(s'|s, a) = \delta(s' - [\cos(\theta_{n+1}), \sin(\theta_{n+1}), \dot{\theta}_{n+1}])$

$$\dot{\theta}_{n+1} = \min \left(\max \left(\dot{\theta}_n + \left(\frac{3g}{2l} \sin(\theta) + \frac{3\tau}{ml^2} \right) \Delta t, -8 \right), 8 \right)$$

$$\theta_{n+1} = \theta_n + \dot{\theta}_n \Delta t$$

gravity motor's influence

- A **reward function** $r(s, a) = -(\theta^2 + 0.1\dot{\theta}^2 + 0.001\tau^2)$

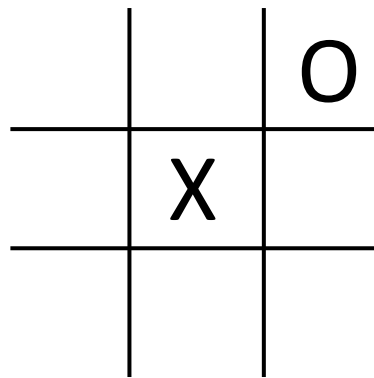


Markov Decision Processes

A **finite** MDP is specified by

- A set of **states** $\mathcal{S}$
- A set of **actions** $\mathcal{A}$
- **Initial state distribution** $P(s_0)$
- **Transition probability** $P(s'|s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$
- A **reward function** $r(s, a)$

Example: Tic-Tac-Toe



- A set of **states** $\mathbb{S} = \{0,1,2\}^9$
 - A set of **actions** $\mathbb{A} = \{0,1,2,3,4,5,6,7,8\}$
 - **Initial state distribution** $P(s_0) = \delta(\mathbf{0}_9)$
 - **Transition probability** $P(s'|s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$
→ depends on opponent
 - A **reward function** $r(s, a)$., e,g 1 if win, 0 otherwise
OR +1 for winning, 0 for drawing, -1 for losing
- Focus for now on finite MDPs

Value Functions in MDPs

- Fix deterministic policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$
 - Deterministic: one input, one fixed output
 - Stochastic: one input, a distribution of possible outputs
- MDP becomes a Markov chain with transition probabilities

$$P(S_{t+1} = s' | S_t = s) = P(s' | s, \pi(s))$$

- Value function $V^\pi(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(S_t, \pi(S_t)) | S_0 = s]$

discount factor

If we start in state s and always follow policy π , how much total discounted reward should we expect to receive?

- The discount factor (折扣因子) satisfies $0 < \gamma < 1$.

Reward: immediate consequence of taking a ,

Value: long-term consequence of following π .

A useful intuition for the effective planning horizon (有效规划范围) is

$$H \approx \frac{1}{1 - \gamma}.$$

For example:

$$\gamma = 0.9 \Rightarrow H \approx 10,$$

$$\gamma = 0.99 \Rightarrow H \approx 100.$$

This is an approximation, not a strict cutoff.

4. Why is there an expectation?

Even if the policy is deterministic, the next state may be stochastic:

$$S_{t+1} \sim P(\cdot \mid S_t, \pi(S_t)).$$

Therefore, starting from the same state s , different trajectories may occur:

$$s \rightarrow s'_1 \rightarrow s'_2 \rightarrow \dots$$

or

$$s \rightarrow \tilde{s}_1 \rightarrow \tilde{s}_2 \rightarrow \dots .$$

Each trajectory can produce a different return. The value function averages over these possible trajectories:

$$V^\pi(s) = \sum_{\text{trajectories}} P(\text{trajectory} \mid s, \pi) \times G(\text{trajectory}).$$

The randomness may come from:

- Stochastic environment dynamics
- A stochastic opponent
- Sensor or process noise
- A stochastic policy, if policies are generalized beyond the deterministic case

Recursive Definition of Value Functions

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s \right]$$

linearity $= \mathbb{E} [r(s_0, \pi(s_0)) \mid s_0 = s] + \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, \pi(s_t)) \mid s_0 = s \right]$

index shift $= r(s, \pi(s)) + \mathbb{E} \left[\gamma \sum_{t=0}^{\infty} \gamma^t r(s_{t+1}, \pi(s_{t+1})) \mid s_0 = s \right]$

stationarity $= r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim P(s', s, \pi(s))} \left[\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_{t+1}, \pi(s_{t+1})) \mid s_t = s' \right] \right]$

def. cond. expec. $= r(s, \pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) V^\pi(s') \quad \underbrace{\left(\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_{t+1}, \pi(s_{t+1})) \mid s_t = s' \right] \right)}_{V^\pi(s')}$

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) V^\pi(s')$$

Value Function in Matrix-Vector Notation

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

→ Finite number of states n :

$$V^\pi = \begin{bmatrix} V^\pi(1) \\ \vdots \\ V^\pi(n) \end{bmatrix}, r^\pi = \begin{bmatrix} r(1, \pi(1)) \\ \vdots \\ r(n, \pi(n)) \end{bmatrix},$$

go to state n

$$T^\pi = \begin{bmatrix} P(1|1, \pi(1)) & \dots & P(n|1, \pi(1)) \\ \vdots & \ddots & \vdots \\ P(1|n, \pi(n)) & \dots & P(n|n, \pi(n)) \end{bmatrix}$$

← start from state 1

3. Transition matrix

After fixing policy π , define:

$$T_{ij}^{\pi} = P(S_{t+1} = j \mid S_t = i, A_t = \pi(i)).$$

The complete transition matrix is:

$$T^{\pi} = \begin{bmatrix} P(1|1, \pi(1)) & \cdots & P(n|1, \pi(1)) \\ \vdots & \ddots & \vdots \\ P(1|n, \pi(n)) & \cdots & P(n|n, \pi(n)) \end{bmatrix}$$

Interpretation:

- Row i : current state is i
- Column j : next state is j
- Entry (i, j) : probability of transitioning from i to j

4. Each row sums to one

For a fixed current state i , the next state must be one of the n states:

$$\sum_{j=1}^n P(j|i, \pi(i)) = 1.$$

Therefore:

$$\sum_{j=1}^n T_{ij}^{\pi} = 1.$$

A matrix whose entries are non-negative and whose rows sum to one is called a row-stochastic matrix (行随机矩阵).

For example:

$$T^{\pi} = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix}.$$

Each row sums to one:

$$0.7 + 0.3 = 1, \quad 0.2 + 0.8 = 1.$$

5. Matrix-vector multiplication

Consider:

$$T^\pi V^\pi.$$

Its i -th component is:

$$(T^\pi V^\pi)_i = \sum_{j=1}^n T_{ij}^\pi V^\pi(j).$$

Substitute the definition of T_{ij}^π :

$$(T^\pi V^\pi)_i = \sum_{j=1}^n P(j|i, \pi(i)) V^\pi(j).$$

This is exactly **the expected value of the next state**:

$$(T^\pi V^\pi)_i = \mathbb{E}[V^\pi(S_{t+1}) \mid S_t = i]$$

Therefore, multiplication by T^π calculates a probability-weighted average of the next-state values.

6. Bellman equation in matrix form

For state i , the scalar Bellman equation is:

$$V^\pi(i) = r(i, \pi(i)) + \gamma \sum_{j=1}^n P(j|i, \pi(i)) V^\pi(j).$$

Writing all n equations simultaneously gives: Value of every state = Immediate reward + Discounted expected next-state value

$$V^\pi = r^\pi + \gamma T^\pi V^\pi$$

This single matrix equation represents n scalar equations.

7. Two-state example

Suppose:

$$T^\pi = \begin{bmatrix} 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix}, \quad r^\pi = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \gamma = 0.9.$$

Let:

$$V^\pi = \begin{bmatrix} V^\pi(1) \\ V^\pi(2) \end{bmatrix}.$$

The matrix Bellman equation is:

$$\begin{bmatrix} V^\pi(1) \\ V^\pi(2) \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 0.9 \begin{bmatrix} 0.5 & 0.5 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} V^\pi(1) \\ V^\pi(2) \end{bmatrix}.$$

Perform the matrix multiplication:

$$T^\pi V^\pi = \begin{bmatrix} 0.5V^\pi(1) + 0.5V^\pi(2) \\ 0.2V^\pi(1) + 0.8V^\pi(2) \end{bmatrix}.$$

Therefore, the matrix equation represents:

$$V^\pi(1) = 1 + 0.9 (0.5V^\pi(1) + 0.5V^\pi(2)),$$

$$V^\pi(2) = 2 + 0.9 (0.2V^\pi(1) + 0.8V^\pi(2)).$$

So matrix notation does not introduce a new idea. It simply compresses all state equations into one expression.

8. Dimension check

Always check the dimensions:

$$T^\pi \in \mathbb{R}^{n \times n}, \quad V^\pi \in \mathbb{R}^{n \times 1}.$$

Therefore:

$$T^\pi V^\pi \in \mathbb{R}^{n \times 1}.$$

Then:

$$r^\pi + \gamma T^\pi V^\pi$$

is valid because both terms are $n \times 1$ vectors.

Solving for Value Functions

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$



$$V^\pi = r^\pi + \gamma T^\pi V^\pi$$

Compute V^π by solving
linear system

$$V^\pi = (I - \gamma T^\pi)^{-1} r^\pi$$

← identity matrix

Compute V^π via fixed
point iteration

Fixed Point Iterations for Value Functions

1. Initialize V_0^π
2. **For** $t = 1:T$ do
3. $V_t^\pi = r^\pi + \gamma T^\pi V_{t-1}^\pi$

→ can show:

$$\|V_t^\pi - V^\pi\|_\infty \leq \gamma^t \|V_0^\pi - V^\pi\|_\infty$$

contraction 压缩映射
error shrink

→ computational advantages of fixed point iteration

Probability averaging does not enlarge error + discount shrinks error = convergence

What is the fixed-point iteration convergence proof?

The proof shows that repeatedly applying the policy Bellman update

$$V_t^\pi = r^\pi + \gamma T^\pi V_{t-1}^\pi$$

will converge to the true value function V^π , regardless of the initial estimate V_0^π .

The final conclusion is:

$$\|V_t^\pi - V^\pi\|_\infty \leq \gamma^t \|V_0^\pi - V^\pi\|_\infty \rightarrow 0$$

because $0 \leq \gamma < 1$.

1. Why is it called a fixed point?

Define the Bellman operator (贝尔曼算子):

$$B^{\pi}(V) = r^{\pi} + \gamma T^{\pi} V.$$

Then the true value function satisfies

$$V^{\pi} = B^{\pi}(V^{\pi}).$$

A fixed point (不动点) of a function f is a value x^* satisfying

$$f(x^*) = x^*.$$

5. Convergence guarantee

The slide gives:

$$\|V_t^\pi - V^\pi\|_\infty \leq \gamma^t \|V_0^\pi - V^\pi\|_\infty$$

The infinity norm (无穷范数) is:

$$\|x\|_\infty = \max_i |x_i|.$$

Therefore,

$$\|V_t^\pi - V^\pi\|_\infty = \max_s |V_t^\pi(s) - V^\pi(s)|.$$

It measures the largest error among all states.

For example, if

$$V_t^\pi - V^\pi = \begin{bmatrix} 0.2 \\ -0.7 \\ 0.4 \end{bmatrix},$$

then

$$\|V_t^\pi - V^\pi\|_\infty = 0.7.$$

8. Why use iteration instead of matrix inversion?

Page 22 gives the exact expression:

$$V^\pi = (I - \gamma T^\pi)^{-1} r^\pi.$$

But explicitly computing a matrix inverse is expensive:

$$O(n^3)$$

time for a dense $n \times n$ matrix, together with roughly $O(n^2)$ memory.

One fixed-point iteration requires a matrix-vector multiplication:

$$T^\pi V.$$

For a dense transition matrix, this costs

$$O(n^2)$$

per iteration.

Robotic and gridworld transition matrices are frequently sparse (稀疏的): each state can transition to only a few next states. If there are m nonzero transitions, one iteration can cost approximately

$$O(m),$$

which may be much smaller than $O(n^2)$.

The trade-off is:

- **Linear-system solution: high cost but directly obtains the exact numerical solution.**
- **Fixed-point iteration: lower cost per iteration but needs repeated updates and returns an approximation after finite time.**

Fixed Point Iteration Convergence Proof

$$V_t^\pi = v^\pi + \gamma T^\pi V_{t-1}^\pi =: B^\pi V_{t-1}^\pi \rightarrow \text{fixed point } V^\pi = B^\pi V^\pi$$

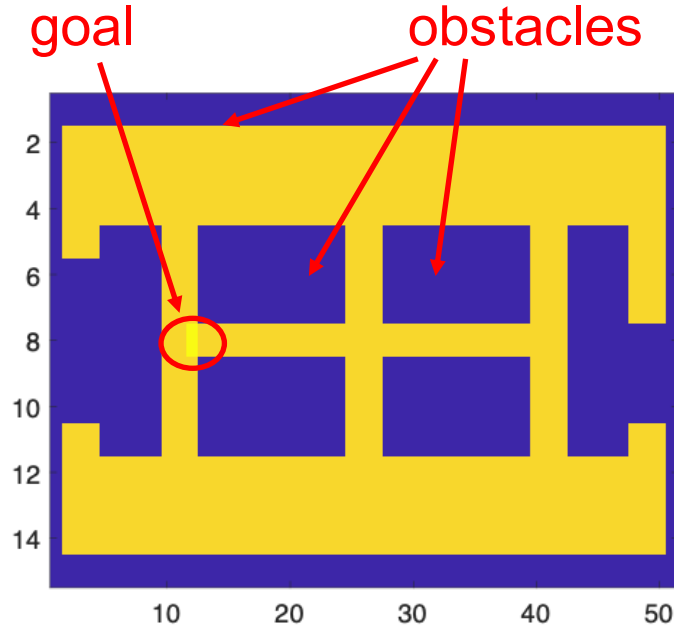
Claim: B^π is contraction: $\|B^\pi V - B^\pi V'\|_\infty \leq \underbrace{\gamma}_{\leq 1} \|V - V'\|_\infty$ $\|V\|_\infty = \max_i |V_i|$

$$\|B^\pi V - B^\pi V'\|_\infty = \|\cancel{v^\pi} + \gamma T^\pi V - \cancel{v^\pi} - \gamma T^\pi V'\|_\infty = \gamma \|T^\pi V - T^\pi V'\|_\infty$$

$$= \gamma \|T^\pi(V - V')\|_\infty = \gamma \max_s \left| \underbrace{\sum_{s'} p(s' | s, \pi(s))}_{=1} \underbrace{(V(s') - V'(s'))}_{\leq \|V - V'\|_\infty} \right| \leq \gamma \|V - V'\|_\infty$$

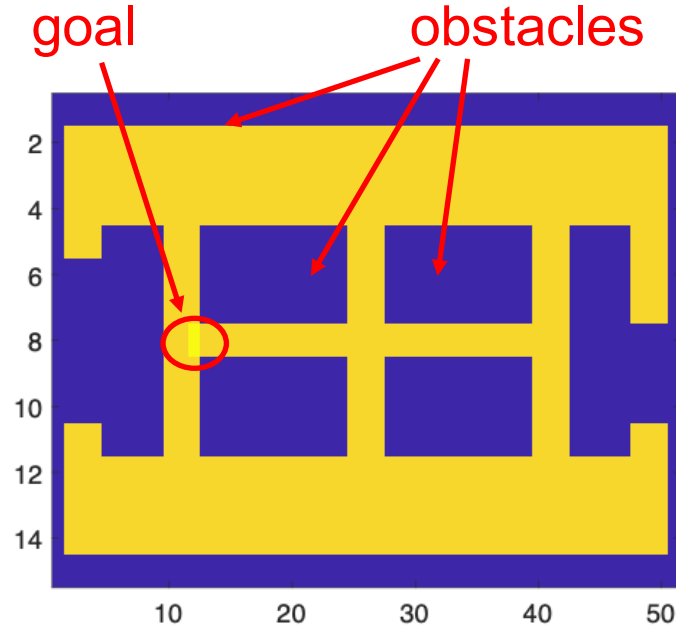
$$\Rightarrow \|V_t - V^\pi\|_\infty = \|B^\pi V_{t-1} - B^\pi V^\pi\|_\infty \leq \gamma \|V_{t-1} - V^\pi\|_\infty \\ \leq \gamma^t \|V_0 - V^\pi\|_\infty$$

Gridworld Example

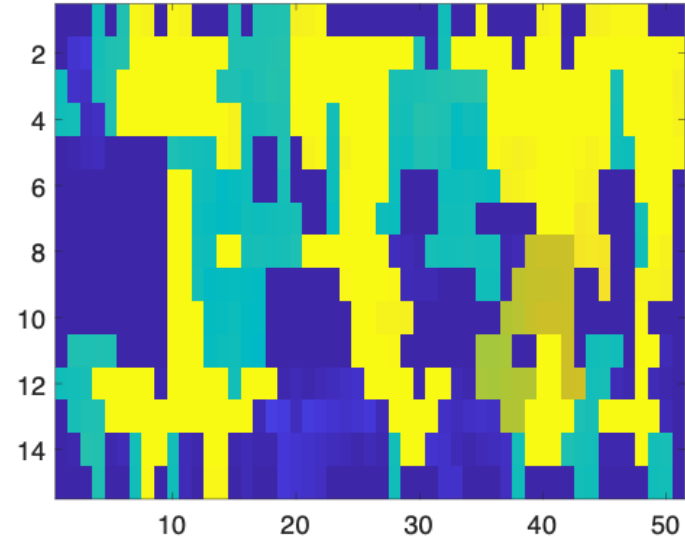


actions: left, right, up, down
move into obstacle: reward -10
move into goal: reward 1
this example: $15 \times 51 = 765$ states
discount factor: 0.99

Gridworld Example



value function of random policy



<https://ch.mathworks.com/matlabcentral/fileexchange/25786-markov-decision-processes-mdp-toolbox>

Picking Actions Given Value Functions

- How should we pick an action given a state?

$$\arg \max_a r(s, a) ?$$

arg max returns the action that gives the largest value

→ does not take into account future rewards

- If we knew V , we could chose **One-step lookahead**

$$\arg \max_a \left(r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right)$$

→ greedy w.r.t. V 根据 V 选择最优动作

the policy greedy with respect to V 关于 V 的贪心策略

Connecting Value Functions and Policies

Every value function induces a policy

Value function $V^\pi(s)$

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

Greedy policy w.r.t. V

$$\pi_V(s) = \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

Every policy induces a value function

Bellman's Identity

Theorem (Bellman):

Policy optimal $\Leftrightarrow$ greedy w.r.t. its induced value function

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

→ Planning problem: find value function satisfying

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

Policy iteration

Value iteration

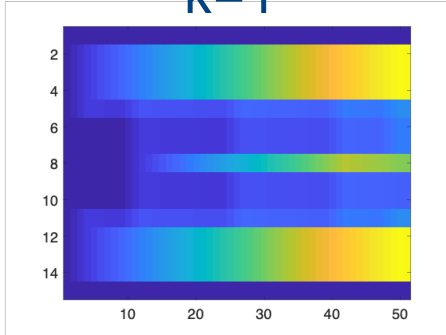
Policy Iteration

1. Start with arbitrary policy π
 2. **While** not converged **do**
 3. Compute value function V^π
 4. Compute greedy policy π_g w.r.t. V^π 
 5. Set $\pi \leftarrow \pi_g$
- Given the future values produced by the current policy, is there a better first action in this state?

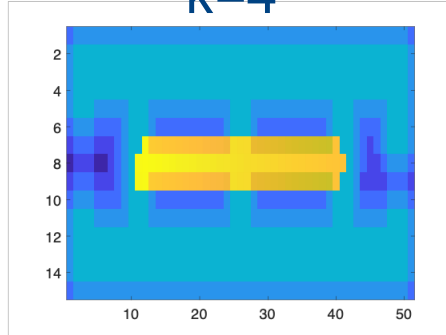
Convergence to optimal policy in $\mathcal{O}\left(\frac{|\mathcal{S}|^2|\mathcal{A}|}{1-\gamma}\right)$ iterations [Ye '10]!

Policy Iteration Demo

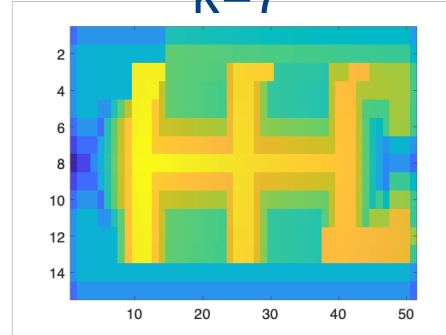
k=1



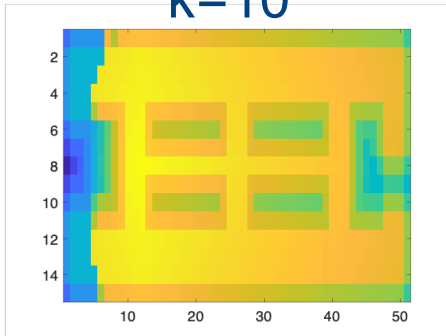
k=4



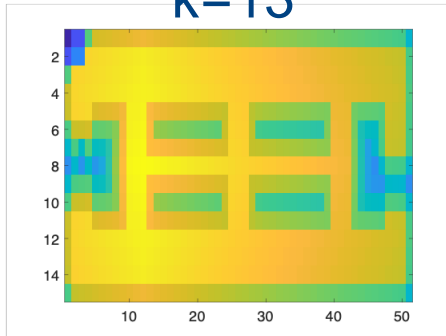
k=7



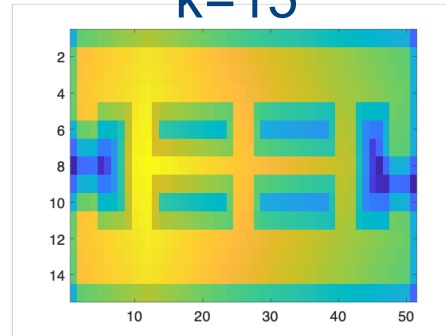
k=10



k=13



k=15



Value Iteration

- Start from value function instead of policy
- Recall: $V^*(s) = \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')$

Approximate using value
for t step rollout V_t

$$V_0(s) = \max_a r(s, a)$$

$$V_1(s) = \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_0(s')$$

$$V_2(s) = \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_1(s')$$

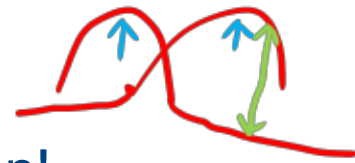
→ compute V^* using **dynamic programming** 动态规划

Value Iteration continued

1. Initialize $V_0(s) = \max_a r(s, a)$
2. **Do**
3. Define $Q_t(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_{t-1}(s')$
4. Find $V_t(s) = \max_a Q_t(s, a)$
5. **While** $\|V_t - V_{t-1}\|_\infty = \max_x |V_t(s) - V_{t-1}(s)| \leq \epsilon$
6. Compute greedy policy π_g w.r.t. V_t

Guaranteed to converge to ϵ -optimal policy!

Convergence of Value Iteration



- Key observation: Bellman update is a contraction!

$$(B^*V)(s) = \max_a \underbrace{r(s,a) + \gamma \sum_{s'} p(s'|s,a) V(s')}_{Q(s,a)}$$

$$\|B^*V - B^*V'\|_\infty = \max_s | \max_a (B^*V)(s) - (B^*V')(s) | = \max_s | \max_a Q(s,a) - \max_{a'} Q(s,a') |$$

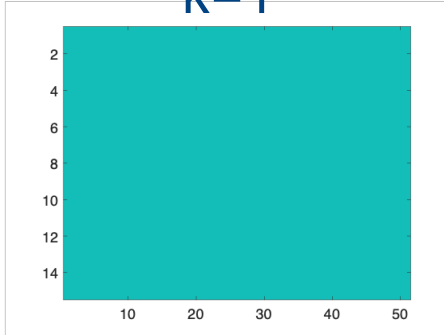
$$\leq \max_{s,a} | Q(s,a) - Q'(s,a) |$$

$$\leq \max_{s,a} | \cancel{r(s,a)} + \gamma \sum_{s'} p(s'|s,a) V(s') - \cancel{r(s,a)} - \gamma \sum_{s'} p(s'|s,a) V'(s') |$$

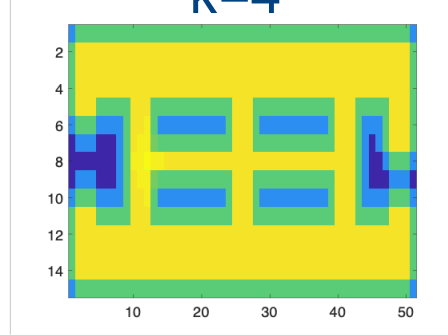
$$\leq \gamma \max_{s,a} | \underbrace{\sum_{s'} p(s'|s,a)}_{=1} \underbrace{(V(s') - V'(s'))}_{\leq \|V - V'\|_\infty} | \leq \gamma \|V - V'\|_\infty$$

Value Iteration Demo

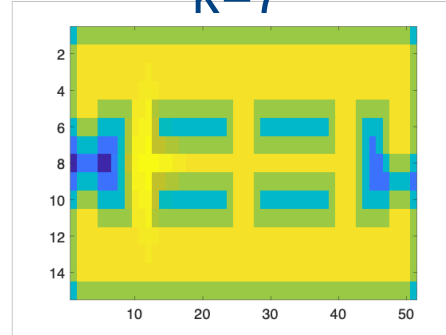
k=1



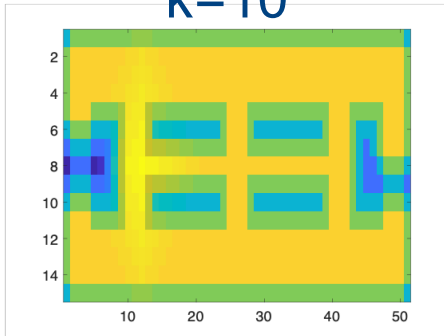
k=4



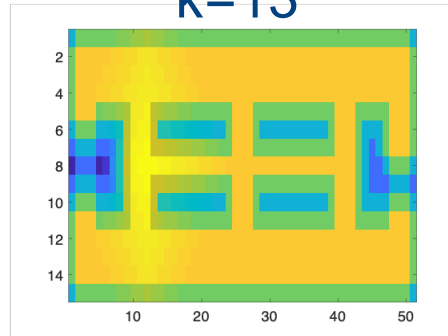
k=7



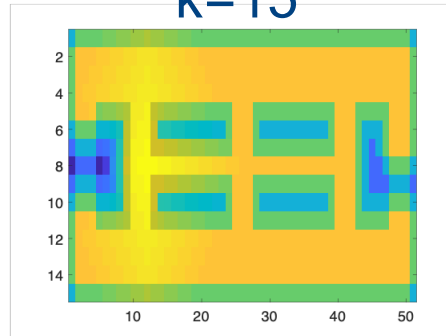
k=10



k=13



k=15



Value vs. Policy Iteration

- Policy iteration
 - Converges to exact solution in polynomial number of iterations
→ cubic complexity per iteration $\mathcal{O}(|S|^3)$
- Value iteration:
 - Finds ϵ -optimal solution in polynomial number of iterations
→ complexity per iteration $\mathcal{O}(|S|^2|A|)$

What works better in practice? → depends on application

Comparison	Policy iteration	Value iteration
Main object updated	Policy π	Value function V
Main operation	Fully evaluate the current policy, then improve it	Perform one Bellman optimality update
Cost per iteration	Higher	Lower
Dense complexity per iteration	$O(S^2)$	$O(S)$
Typical number of iterations	Fewer	More
Policy evaluation	Complete or highly accurate	No complete evaluation of an intermediate policy
Finite-time result	Exact optimal policy, assuming exact evaluation	Approximately optimal policy
Stopping condition	The policy no longer changes	$\ V_t - V_{t-1}\ _\infty \leq \epsilon$
Main advantage	Usually needs few policy improvements	Each iteration is relatively cheap
Main disadvantage	Solving the linear system is expensive	May require many iterations
Suitable situation	Small or medium state space; high accuracy needed	Large or sparse MDP; approximation acceptable

The central difference is:

Policy iteration = expensive iterations + usually fewer iterations

Value iteration = cheaper iterations + usually more iterations

Recap Quiz



Summary

- Policy evaluation via matrix inverse or fixed point iteration
- Planning in finite MDPs via policy or value iteration
- Policy Evaluation:
 - Start with random policy π
 - Compute exact value function V^π
 - Select greedy policy w.r.t. V^π
- Value iteration:
 - Solve Bellman equation using dynamic programming:
$$V_t(s) = \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_{t-1}(s')$$
 - When converged, select greedy policy w.r.t. V_t

Homework

- Set up an environment for a problem of your choice
- EE6108 students: run value iteration
- CEG5306 students: run policy iteration
- Illustrate the resulting policy (video, image, etc.)
- Write 1-2 sentences why your submission should be highlighted
- Submission deadline: Monday 11am

Homework

- Not graded
- Homework questions will be part of graded assignments
- Selected submissions will be presented next lecture
- Presenting students will receive bonus points in assignment

References

- Y. Ye: The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Mathematics of Operations Research*, 36(4), 2011.